Denau Hermanus

ST10497747

PROG5121

Assignment PART 2

[denauhermanus881-hash/PROG5121_Part2]

Login.java

```
public class Login {
    private String username;
    private String password;
    private String firstName;
    private String lastName;
    private String cellPhoneNumber;

    public Login() {
        this.username = "";
        this.password = "";
        this.firstName = "";
        this.lastName = "";
        this.cellPhoneNumber = "";
    }

    public Login(String username, String password, String firstName, String
lastName, String cellPhoneNumber) {
        this.username = username;
        this.password = password;
        this.firstName = firstName;
        this.lastName = lastName;
        this.cellPhoneNumber = cellPhoneNumber;
    }

    public boolean checkUserName() {
        return username != null && username.contains("_") && username.length() <=
5;
    }

    public boolean checkPasswordComplexity() {
        if (password == null || password.length() < 8) {
            return false;
        }
```

```
        boolean hasCapital = false;
        boolean hasNumber = false;
        boolean hasSpecial = false;

        for (int i = 0; i < password.length(); i++) {
            char c = password.charAt(i);
            if (Character.isUpperCase(c)) hasCapital = true;
            else if (Character.isDigit(c)) hasNumber = true;
            else if (!Character.isLetterOrDigit(c)) hasSpecial = true;
        }
        return hasCapital && hasNumber && hasSpecial;
    }

    public boolean checkCellPhoneNumber() {
        return cellPhoneNumber != null && cellPhoneNumber.startsWith("+") &&
cellPhoneNumber.length() <= 12;
    }

    public String registerUser() {
        if (!checkUserName()) {
            return "Username is incorrectly formatted";
        }
        if (!checkPasswordComplexity()) {
            return "Password is not correctly formatted";
        }
        if (!checkCellPhoneNumber()) {
            return "Cell phone number is incorrectly formatted";
        }
        return "User successfully registered";
    }

    public boolean loginUser(String enteredUsername, String enteredPassword) {
        return this.username.equals(enteredUsername) &&
this.password.equals(enteredPassword);
    }

    public String returnLoginStatus(boolean isLoggedIn) {
        if (isLoggedIn) {
            return "Welcome " + firstName + " " + lastName + ", it is great to
see you again.";
        } else {
            return "Username or password incorrect, please try again";
        }
```

```
    }

    // Getters and Setters
    public void setUsername(String username) { this.username = username; }
    public void setPassword(String password) { this.password = password; }
    public void setCellPhoneNumber(String cellPhoneNumber) { this.cellPhoneNumber
= cellPhoneNumber; }
    public String getUsername() { return username; }
    public String getPassword() { return password; }
}
```

Message.java

```
import java.io.FileWriter;
import java.io.IOException;
import java.util.Random;

public class Message {
    private String messageID;
    private int messageNumber;
    private String recipientCell;
    private String messageContent;
    private String messageHash;
    private String messageStatus;

    private static Message[] storageArray = new Message[100];
    private static int messageCount = 0;

    public Message(int messageNumber, String recipientCell, String
messageContent) {
```

```
        this.messageID = generateID();
        this.messageNumber = messageNumber;
        this.recipientCell = recipientCell;
        this.messageContent = messageContent;
        this.messageHash = createMessageHash();
        this.messageStatus = "Pending";
    }

    private String generateID() {
        Random r = new Random();
        long num = 1000000000L + (long)(r.nextDouble() * 9000000000L);
        return String.valueOf(num);
    }

    public boolean checkMessageID() {
        return messageID != null && messageID.length() <= 10;
    }

    public String checkRecipientCell() {
        if (recipientCell != null && recipientCell.startsWith("+") &&
recipientCell.length() <= 12) {
            return "Cell phone number successfully captured.";
        } else {
            return "Cell phone number is incorrectly formatted or does not
contain an international code. Please correct the number and try again.";
        }
    }

    public String checkMessageLength() {
        if (messageContent == null) {
            return "Message ready to send.";
        }
        int length = messageContent.length();
        if (length <= 250) {
            return "Message ready to send.";
        } else {
            int excess = length - 250;
            return "Message exceeds 250 characters by " + excess + " characters;
please reduce the size.";
        }
    }

    public String createMessageHash() {
```

```
        if (messageContent == null || messageContent.trim().isEmpty()) {
            return messageID.substring(0, 2).toUpperCase() + ":" + messageNumber
+ ":NONAME";
        }
        String[] words = messageContent.trim().split("\\s+");
        String firstWord = words[0];
        String lastWord = words[words.length - 1];

        String prefix = messageID.substring(0, 2);
        return (prefix + ":" + messageNumber + ":" + firstWord +
lastWord).toUpperCase();
    }

    public static boolean storeMessage(Message message) {
        if (messageCount < storageArray.length) {
            storageArray[messageCount] = message;
            messageCount++;
            return true;
        } else {
            return false;
        }
    }

    public void saveToFile() {
        String json = "{\n" +
                      "  \"messageID\": \"" + messageID + "\",\n" +
                      "  \"messageNumber\": " + messageNumber + ",\n" +
                      "  \"recipientCell\": \"" + recipientCell + "\",\n" +
                      "  \"messageContent\": \"" + messageContent + "\",\n" +
                      "  \"messageHash\": \"" + messageHash + "\",\n" +
                      "  \"status\": \"" + messageStatus + "\"\n" +
                      "}\n";

        try {
            FileWriter writer = new FileWriter("messages.json", true);
            writer.write(json);
            writer.close();
        } catch (IOException e) {
            System.out.println("Error saving file");
        }
    }

    public String SentMessage(int choice) {
```

```
        if (choice == 1) {
            storeMessage(this);
            saveToFile();
            this.messageStatus = "Sent";
            return "Message successfully sent.";
        } else if (choice == 2) {
            this.messageStatus = "Disregarded";
            return "Press 0 to delete the message.";
        } else if (choice == 3) {
            storeMessage(this);
            saveToFile();
            this.messageStatus = "Stored";
            return "Message successfully stored.";
        }
        return "Invalid selection.";
    }

    public static String printMessages() {
        StringBuilder sb = new StringBuilder();
        for (int i = 0; i < messageCount; i++) {
            Message m = storageArray[i];
            sb.append("Message ID: ").append(m.getMessageID()).append("\n")
              .append("Message Hash: ").append(m.getMessageHash()).append("\n")
              .append("Recipient: ").append(m.getRecipientCell()).append("\n")
              .append("Message: ").append(m.getMessageContent()).append("\n")
              .append("Status: ").append(m.messageStatus).append("\n\n");
        }
        return sb.toString();
    }

    public static int returnTotalMessagess() {
        return messageCount;
    }

    // Getters and Setters
    public String getMessageID() { return messageID; }
    public String getMessageHash() { return messageHash; }
    public String getRecipientCell() { return recipientCell; }
    public String getMessageContent() { return messageContent; }
    public void setMessageStatus(String status) { this.messageStatus = status; }
}
```

Main.java

```
import javax.swing.JOptionPane;

public class Main {
    public static void main(String[] args) {
        JOptionPane.showMessageDialog(null, "Welcome to QuickChat.");

        // --- 1. REGISTRATION PHASE ---
        String regUsername = JOptionPane.showInputDialog("Register - Enter
username (must contain '_' and <= 5 chars):");
        String regPassword = JOptionPane.showInputDialog("Register - Enter
password (>= 8 chars, 1 capital, 1 number, 1 special):");
        String regFirst = JOptionPane.showInputDialog("Register - Enter your
First Name:");
        String regLast = JOptionPane.showInputDialog("Register - Enter your Last
Name:");
        String regCell = JOptionPane.showInputDialog("Register - Enter cell phone
number (+ code, max 12 chars):");

        Login login = new Login(regUsername, regPassword, regFirst, regLast,
regCell);
        String regStatus = login.registerUser();
        JOptionPane.showMessageDialog(null, regStatus);

        if (!regStatus.equals("User successfully registered")) {
            return;
        }
```

```
        // --- 2. LOGIN PHASE ---
        String loginUser = JOptionPane.showInputDialog("Login - Enter your
username:");
        String loginPass = JOptionPane.showInputDialog("Login - Enter your
password:");

        boolean isLoggedIn = login.loginUser(loginUser, loginPass);
        JOptionPane.showMessageDialog(null, login.returnLoginStatus(isLoggedIn));

        if (!isLoggedIn) {
            return;
        }

        // --- 3. MAIN MENU LOOP (PART 2) ---
        boolean running = true;

        while (running) {
            String act = JOptionPane.showInputDialog(
                "Please choose an option:\n" +
                "1) Send Messages\n" +
                "2) Show recently sent messages\n" +
                "3) Quit"
            );

            if (act == null || act.equals("3")) {
                JOptionPane.showMessageDialog(null, "Goodbye!");
                running = false;
                break;
            }

            if (act.equals("1")) {
                // Prompt user how many messages they wish to enter
                String numInput = JOptionPane.showInputDialog("Enter how many
messages you wish to send:");
                if (numInput == null) continue;

                int totalMessages;
                try {
                    totalMessages = Integer.parseInt(numInput);
                } catch (NumberFormatException e) {
                    JOptionPane.showMessageDialog(null, "Invalid number
entered.");
                    continue;
```

```
                }

                // Loop for the set number of messages
                for (int i = 0; i < totalMessages; i++) {
                    String cell = JOptionPane.showInputDialog("Message " + (i +
1) + " - Enter Recipient cell (+ code, max 12 chars):");
                    String content = JOptionPane.showInputDialog("Message " + (i
+ 1) + " - Enter message content (max 250 characters):");

                    Message msg = new Message(i, cell, content);

                    // Validate Cell
                    String cellValidation = msg.checkRecipientCell();
                    if (!cellValidation.startsWith("Cell phone number
successfully")) {
                        JOptionPane.showMessageDialog(null, cellValidation);
                        i--; // Retry this message iteration
                        continue;
                    }

                    // Validate Length
                    String lengthValidation = msg.checkMessageLength();
                    if (!lengthValidation.equals("Message ready to send.")) {
                        JOptionPane.showMessageDialog(null, lengthValidation);
                        i--; // Retry this message iteration
                        continue;
                    }

                    // Sub-menu for Send / Discard / Store
                    String choiceStr = JOptionPane.showInputDialog(
                        "Message validated successfully!\n" +
                        "Choose an option:\n" +
                        "1. Send Message\n" +
                        "2. Disregard Message\n" +
                        "3. Store Message to send later"
                    );

                    if (choiceStr != null) {
                        int choice = Integer.parseInt(choiceStr);
                        String result = msg.SentMessage(choice);

                        if (choice == 2) {
```

```
                            String del = JOptionPane.showInputDialog(result); //
"Press 0 to delete the message."
                            if (del != null && del.equals("0")) {
                                JOptionPane.showMessageDialog(null, "Message
successfully deleted.");
                            } else {
                                JOptionPane.showMessageDialog(null, "Message
status changed to Disregarded.");
                            }
                        } else {
                            JOptionPane.showMessageDialog(null,
                                "--- Message Details ---\n" +
                                "Message ID: " + msg.getMessageID() + "\n" +
                                "Message Hash: " + msg.getMessageHash() + "\n" +
                                "Recipient: " + msg.getRecipientCell() + "\n" +
                                "Message: " + msg.getMessageContent() + "\n\n" +
                                result
                            );
                        }
                    }
                }

                // Display total accumulated messages sent/stored
                JOptionPane.showMessageDialog(null, "Total messages processed: "
+ Message.returnTotalMessagess() + "\n\n" + Message.printMessages());

            } else if (act.equals("2")) {
                JOptionPane.showMessageDialog(null, "Coming Soon.");
            }
        }
    }
}
```

LoginTest.java

```java
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

public class LoginTest {

    @Test
    public void testCheckUserNameValid() {
        Login login = new Login();
        login.setUsername("kyl_1");
        assertTrue(login.checkUserName());
    }

    @Test
    public void testCheckUserNameInvalid() {
        Login login = new Login();
        login.setUsername("kyleusername");
        assertFalse(login.checkUserName());
    }

    @Test
    public void testCheckPasswordComplexityValid() {
        Login login = new Login();
        login.setPassword("Ch&&t123");
        assertTrue(login.checkPasswordComplexity());
    }
```

```
    @Test
    public void testCheckPasswordComplexityInvalid() {
        Login login = new Login();
        login.setPassword("password");
        assertFalse(login.checkPasswordComplexity());
    }

    @Test
    public void testCheckCellPhoneNumberValid() {
        Login login = new Login();
        login.setCellPhoneNumber("+27821234567");
        assertTrue(login.checkCellPhoneNumber());
    }

    @Test
    public void testCheckCellPhoneNumberInvalid() {
        Login login = new Login();
        login.setCellPhoneNumber("0821234567");
        assertFalse(login.checkCellPhoneNumber());
    }

    @Test
    public void testLoginUserSuccess() {
        Login login = new Login("kyl_1", "Ch&&t123", "Kyle", "Smith",
"+27821234567");
        assertTrue(login.loginUser("kyl_1", "Ch&&t123"));
    }
}
```

MessageTest.java

```
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

public class MessageTest {

    @Test
    public void testCheckMessageLengthValid() {
        Message msg = new Message(0, "+2782123", "Hello world");
        assertEquals("Message ready to send.", msg.checkMessageLength());
    }

    @Test
    public void testCheckMessageLengthInvalid() {
        String longText = "a".repeat(260);
        Message msg = new Message(0, "+2782123", longText);
        assertTrue(msg.checkMessageLength().contains("exceeds 250 characters"));
    }

    @Test
    public void testCheckRecipientCellValid() {
        Message msg = new Message(0, "+2782123", "Test message");
        assertEquals("Cell phone number successfully captured.",
msg.checkRecipientCell());
    }
```

```
    @Test
    public void testCheckRecipientCellInvalid() {
        Message msg = new Message(0, "082123456789", "Test message");
        assertTrue(msg.checkRecipientCell().contains("incorrectly formatted"));
    }

    @Test
    public void testCreateMessageHash() {
        Message msg = new Message(1, "+2782123", "Hi Mike, can you join us for
dinner tonight?");
        String hash = msg.createMessageHash();
        assertNotNull(hash);
        assertTrue(hash.contains("HIMICHAEL") || hash.contains("HITONIGHT"));
    }

    @Test
    public void testStoreMessageArray() {
        Message msg = new Message(0, "+2782123", "Storage test");
        boolean stored = Message.storeMessage(msg);
        assertTrue(stored);
    }
}
```